JNX Learning Platform - Abacus Agent Onboarding

Purpose: This document provides a structured guide for new Abacus AI chat sessions to understand and extend the JNX Learning Platform.

Last Updated: 2024-12-28
Phase: Building Qryx (First Product) - Phase 3 Complete
Status: Production-Ready ✅
First Product: Qryx (Shopify AI Sales Assistant)

🎯 What is JNX Learning Platform?

TL;DR: A centralized AI learning system that collects data from all JNX products (Qryx, Trading Bot, etc.) to enable:
- Pattern recognition
- Performance optimization
- Continuous improvement
- Cross-product learning

Current Products:
- ✅ **Qryx** (Phase 3 Complete) - Shopify AI Sales Assistant with Gemini 2.0 Flash
- 🚧 Trading Bot (Planned)
- 🚧 More products coming...

Architecture:

```
JNX-OS Core → [Product Registry] → [Event Logger] → [API] → [Database]
                    ↓                    ↓
                  Qryx            Trading Bot (Planned)
                  (Active)
```

Key Files:
- `lib/jnx-core/` - Core SDK (Registry, Logger, Types)
- `lib/jnx-products/` - Product configurations
- `app/api/jnx/events/route.ts` - API endpoint
- `MIGRATION_JNX_LEARNING_PLATFORM.sql` - Database schema

🚀 Task: Add New Product (3 Steps)

When to Use This:

- User says: “Integrate [Product Name] with JNX Learning Platform”
- User says: “Add [Product Name] to the learning system”
- User says: “Enable AI learning for [Product Name]”

Step 1: Create Product Configuration

Action: Create file `lib/jnx-products/[product-name]/config.ts`

Template:

```
import { z } from 'zod'
import { defineProduct } from '@/lib/jnx-core/registry'

export default defineProduct({
  id: '[product_id]',              // lowercase, underscores
  name: '[Product Display Name]',  // human-readable
  version: '1.0.0',

  // Define events that this product will log
  events: {
    '[event_name]': {
      schema: z.object({
        // Define event structure
        field1: z.string(),
        field2: z.number().optional(),
      }),
      description: 'What this event tracks'
    }
  },

  // Paths AI must NEVER modify
  protected: [
    'core/*',           // Core business logic
    'api/payment/*',    // Payment processing
    'api/webhooks/*',   // External integrations
  ],

  // Paths AI CAN optimize (with approval)
  optimizable: [
    'prompts/*',        // AI prompts
    'ui/formatting',    // UI improvements
    'performance/*',    // Performance tuning
  ],

  // Optimization targets
  goals: {
    metricName: {
      target: 100,      // Target value
      unit: 'ms'        // Unit (ms, percentage, rating, etc.)
    }
  }
})
```

Example (Trading Bot):

```
import { z } from 'zod'
import { defineProduct } from '@/lib/jnx-core/registry'

export default defineProduct({
  id: 'trading_bot',
  name: 'JNX Trading Bot',
  version: '1.0.0',

  events: {
    'trade_executed': {
      schema: z.object({
        symbol: z.string(),
        action: z.enum(['buy', 'sell']),
        amount: z.number().positive(),
        price: z.number().positive(),
        profit_loss: z.number(),
        strategy: z.string()
      }),
      description: 'Logged when a trade is executed'
    },
    'market_analysis': {
      schema: z.object({
        symbol: z.string(),
        indicators: z.record(z.number()),
        signal: z.enum(['bullish', 'bearish', 'neutral'])
      }),
      description: 'Market analysis results'
    }
  },

  protected: [
    'core/order-execution',
    'core/risk-management',
    'core/balance-check'
  ],

  optimizable: [
    'strategies/entry-timing',
    'strategies/exit-timing',
    'indicators/parameters'
  ],

  goals: {
    profitability: { target: 0.15, unit: 'percentage' },
    drawdown: { target: 0.05, unit: 'percentage' },
    winRate: { target: 0.6, unit: 'percentage' }
  }
})
```

Step 2: Register Product

Action: Edit `lib/jnx-products/index.ts`

Add this line:

```
import './[product-name]/config'
```

Full file example:

```
// Import all product configurations
import './qryx/config'
import './trading-bot/config'     // ← Add this line

// Export registry utilities
export { registry, defineProduct, getProduct } from '@/lib/jnx-core/registry'
export { useProductLogger, useLogEvent } from '@/lib/jnx-core/hooks'
```

Step 3: Use in Code

Action: Add event logging to product code

Template:

```
import { useProductLogger } from '@/lib/jnx-products'

function YourComponent() {
  const logger = useProductLogger('[product_id]')

  const handleEvent = async () => {
    await logger.logEvent('[event_name]', {
      // Event data matching your schema
      field1: 'value',
      field2: 123
    })
  }

  return <button onClick={handleEvent}>Action</button>
}
```

Example (Trading Bot):

```
import { useProductLogger } from '@/lib/jnx-products'

function TradingDashboard() {
  const logger = useProductLogger('trading_bot')

  const executeTrade = async (trade: Trade) => {
    const result = await exchange.placeOrder(trade)

    // Log the trade
    await logger.logEvent('trade_executed', {
      symbol: trade.symbol,
      action: trade.action,
      amount: trade.amount,
      price: result.price,
      profit_loss: calculatePL(result),
      strategy: trade.strategy
    })
  }

  return <button onClick={() => executeTrade(...)}>Execute Trade</button>
}
```

✅ Verification Checklist

After completing the 3 steps, verify:

```
# 1. TypeScript check
cd /home/ubuntu/jnx-os/nextjs_space
yarn tsc --noEmit
# Expected: No errors

# 2. Build check
yarn build
# Expected: Successful build

# 3. Verify product registered (in code)
import { getAllProducts } from '@/lib/jnx-products'
console.log(getAllProducts())
// Expected: Your product appears in list
```

In database (Supabase SQL Editor):

```
-- Check if events are being logged
SELECT
  product_type,
  event_type,
  COUNT(*) as count
FROM product_events
GROUP BY product_type, event_type
ORDER BY product_type;

-- Expected: Your product_type appears with event counts
```

🎨 Best Practices

Event Schema Design

✅ **Good:**

```
events: {
  'user_action': {
    schema: z.object({
      action: z.enum(['click', 'submit', 'cancel']),
      timestamp: z.string(),
      duration_ms: z.number().positive().optional()
    })
  }
}
```

❌ **Bad:**

```
events: {
  'event1': {  // Not descriptive
    schema: z.object({
      data: z.any(),  // Too generic
      stuff: z.string()  // Unclear meaning
    })
  }
}
```

Protected vs Optimizable Paths

Protected (AI can NEVER modify):
- Core business logic
- Payment processing
- Authentication
- Database operations
- Security features
- External API integrations

Optimizable (AI CAN suggest changes):
- AI prompts and templates
- UI formatting and styling
- Performance optimizations
- Caching strategies
- Response formatting
- User experience improvements

Goal Setting

Good Goals (SMART):

```
goals: {
  responseTime: { target: 2000, unit: 'ms' },           // Specific, Measurable
  accuracy: { target: 0.95, unit: 'percentage' },       // Clear target
  userSatisfaction: { target: 4.5, unit: 'rating' }     // Achievable
}
```

Bad Goals:

```
goals: {
  fast: { target: 1, unit: 'yes' },        // Not measurable
  good: { target: 100, unit: 'good' },     // Unclear unit
  best: { target: 999, unit: 'best' }      // Not achievable
}
```

🐛 Troubleshooting

Error: “Product ‘xyz’ not found”

Cause: Product not registered in `lib/jnx-products/index.ts`

Fix:

```
// lib/jnx-products/index.ts
import './xyz/config'  // Add this line
```

Error: “Event type ‘abc’ not defined”

Cause: Event not in product config

Fix:

```
// lib/jnx-products/your-product/config.ts
events: {
  'abc': {  // Add this event
    schema: z.object({ ... })
  }
}
```

Error: Validation Failed

Cause: Event data doesn’t match schema

Fix: Check Zod error message for details:

```
try {
  await logger.logEvent('event_name', data)
} catch (error) {
  console.error('Validation error:', error)
  // Error will show which field failed and why
}
```

Events Not Appearing in Database

Check:
1. Migration executed? Run `MIGRATION_JNX_LEARNING_PLATFORM.sql` in Supabase
2. User authenticated? Events require Clerk authentication
3. API endpoint working? Check `/api/jnx/events` logs

📚 Reference Files

Core SDK

- `lib/jnx-core/types.ts` - All TypeScript types
- `lib/jnx-core/registry.ts` - Product registry system
- `lib/jnx-core/event-logger.ts` - Event logging with PII redaction
- `lib/jnx-core/hooks.ts` - React hooks for easy use

Example Product

- `lib/jnx-products/qryx/config.ts` - Complete Qryx configuration
- `lib/jnx-products/README.md` - Detailed guide with examples

API & Database

- `app/api/jnx/events/route.ts` - API endpoint

- `MIGRATION_JNX_LEARNING_PLATFORM.sql` - Database schema

Documentation

- `JNX_LEARNING_PLATFORM_PHASE1.md` - Complete Phase 1 guide
- `lib/jnx-products/README.md` - Product integration guide

🔄 Workflow for New Chat Session

If user requests new product integration:

1. **Understand the product:**
   - What does it do?
   - What events should be logged?
   - What are optimization goals?
   - What paths are protected?

2. **Follow 3-step process:**
   - Create config file
   - Register in index.ts
   - Add logging to code

3. **Verify:**
   - TypeScript check
   - Build check
   - Database check

4. **Document:**
   - Update this file if patterns change
   - Add to `lib/jnx-products/README.md` if needed

5. **Save checkpoint:**
   - Build successful
   - All tests passing
   - Documentation updated

⚠️ Important Notes

DO:

- ✅ Follow the 3-step process exactly
- ✅ Use Zod for schema validation
- ✅ Keep event names descriptive
- ✅ Set realistic, measurable goals
- ✅ Test TypeScript before committing
- ✅ Verify events in database

DON'T:

- ❌ Modify core SDK files (`lib/jnx-core/*`)

- ❌ Change API endpoint (`app/api/jnx/events/route.ts`)
- ❌ Use `any` types in schemas
- ❌ Skip validation
- ❌ Forget to register in index.ts
- ❌ Log PII without redaction (automatic in SDK)

🎯 Success Criteria

Product integration is complete when:

1. ✅ Config file created in `lib/jnx-products/[product-name]/config.ts`
2. ✅ Product registered in `lib/jnx-products/index.ts`
3. ✅ Event logging added to product code
4. ✅ TypeScript: No errors
5. ✅ Build: Passing
6. ✅ Events visible in database
7. ✅ Checkpoint saved

📞 Support

For new chat sessions:
- Read this file first
- Check `lib/jnx-products/README.md` for examples
- Review `JNX_LEARNING_PLATFORM_PHASE1.md` for architecture
- Look at `lib/jnx-products/qryx/config.ts` for reference

Common Questions:
- “How do I add a new product?” → Follow 3-step process above
- “What events should I log?” → Depends on product, see examples
- “What are protected paths?” → Core logic, payment, auth, security
- “How do I set goals?” → Use SMART criteria (Specific, Measurable, etc.)

Version: 1.0.0
Phase: 1 Complete
Status: Production-Ready ✅
Last Updated: 2024-12-28